

IIC3745: Testing
Tarea 1: Agentic Test Generation
Juan Pablo Sandoval, José Antonio Tapia

Agosto-2026

1. Objetivos

Al finalizar esta tarea, deberán ser capaces de:

- Diseñar un agente basado en LLM para automatizar una tarea de testing.
- Integrar un LLM con herramientas externas para generar pruebas unitarias que cumplan con estándares de cobertura y resistencia a mutaciones.
- Diseñar estrategias iterativas para corregir y mejorar automáticamente una solución generada por IA.
- Analizar críticamente los resultados generados por el modelo y las limitaciones del *prompt engineering* mediante un video explicativo.

2. Parte I - Implementación del Agente LLM

La escritura de pruebas unitarias es una práctica fundamental para asegurar la calidad del software, pero suele ser una tarea repetitiva y costosa en tiempo. Con el reciente avance de los Modelos de Lenguaje Grande (LLMs), existe una oportunidad enorme para automatizar la generación de estas pruebas. Sin embargo, delegar esta tarea a una Inteligencia Artificial presenta un desafío crítico: los LLMs tienden a alucinar dependencias, inventar métodos que no existen en el código original, o generar aserciones triviales que no validan realmente la lógica del programa.

Para abordar este problema, el objetivo central de esta tarea es construir un **agente automatizado**. Este agente deberá coordinar la interacción con el modelo de lenguaje y gestionar de forma autónoma el proceso de creación de pruebas. El verdadero desafío de diseño radicará en cómo el sistema maneja las respuestas imperfectas de la Inteligencia Artificial. Ustedes deberán idear e implementar una estrategia algorítmica que decida cuándo y cómo volver a interactuar con el modelo para procesar, validar y mejorar iterativamente el código generado, asegurando que la suite de pruebas final sea robusta y funcional.

2.1. Requisitos del agente

El agente desarrollado deberá ser capaz de:

- Analizar automáticamente la estructura de un proyecto Python.
- Extraer el código fuente relevante de una clase objetivo.
- Generar código fuente correspondiente a pruebas unitarias en Python interactuando con la API del LLM.
- Ejecutar iterativamente las pruebas generadas.
- Analizar los errores producidos durante la compilación, ejecución o evaluación de métricas.
- Refinar o regenerar las pruebas si es necesario.

- **Requisito Obligatorio:** Escribir la suite de pruebas final en la carpeta de salida, asegurando estrictamente que el archivo **compile y se ejecute sin errores de sintaxis ni fallos en las aserciones**.
- **Time Budget:** Si bien el agente puede realizar múltiples llamadas a la API del LLM, su ejecución no deberá superar los 2 minutos. Para ello, deberá incorporar un temporizador y exportar los tests generados antes de que finalice el tiempo disponible. Además, deberá considerar que la API de Gemini utilizada en esta tarea posee límites de tokens y de llamadas por minuto.

2.2. Input y Output

El agente recibirá tres parámetros desde la línea de comandos:

- `project_folder`: directorio raíz del proyecto Python.
- `class_file`: ruta al archivo que contiene la clase objetivo.
- `output_folder`: directorio donde deberán escribirse los tests generados (ej. `Resultados/<nombre proyecto>/<nombre clase>`).

El agente será ejecutado mediante:

```
python agent.py <file_path> <output_folder>
```

Dentro de la carpeta de salida, debe existir como mínimo, `test_<nombre clase>.py`. Pueden agregar más archivos si es que encuentran necesario, pero solo se evaluará el archivo de test.

Además de generar el archivo de pruebas, el agente debe calcular y exportar las métricas finales obtenidas obligatoriamente en un archivo llamado `metrics.json` dentro de la carpeta de salida.

El formato de este archivo debe ser estrictamente un objeto JSON con las siguientes llaves (los valores numéricos son solo de ejemplo):

```
{
  "line_coverage": 0.85,
  "branch_coverage": 0.55,
  "mutation_score": 0.60
}
```

2.3. Proyectos de Prueba y Métricas Objetivo

Junto con el enunciado se entregará un conjunto de proyectos y clases de prueba públicos. Este set está diseñado para que validen localmente si su agente es capaz de generar pruebas que no solo compilen, sino que logren buena cobertura, matar mutantes y resistencia frente a escenarios diversos.

A continuación se detalla la lista de proyectos y archivos públicos que deben evaluar:

```
Public_Projects/
|-- blackjack/
|   |-- base.py
|   |-- dealer.py
|   |-- judger.py
|-- gin_rummy/
|   |-- base.py
|   |-- action_event.py
|   |-- dealer.py
|-- mahjong/
|   |-- player.py
|   |-- dealer.py
|   |-- game.py
|-- stock4/
|   |-- tabeformat.py
|   |-- structure.py
|   |-- validate.py
|-- svm/
|   |-- base.py
|   |-- svm.py
|-- tree/
```

```
| |-- base.py
| |-- tree.py
|-- fuzzywuzzy/
| |-- fuzz.py
| |-- string_processing.py
| |-- StringMatcher.py
| |-- utils.py
```

La evaluación final se realizará sobre este set, además de un conjunto diferente de clases y proyectos. Para obtener el puntaje completo en la evaluación automatizada de cada archivo, los tests generados por su agente deberán alcanzar los siguientes umbrales mínimos al ser evaluados:

- **Line Coverage:** $\geq 80\%$.
- **Branch Coverage:** $\geq 50\%$.
- **Mutation Score:** $\geq 50\%$.

2.4. Herramientas disponibles y Stack Tecnológico

El desarrollo debe adherirse estrictamente al siguiente entorno:

- **Lenguaje:** Python 3.14.
- **Framework de Pruebas:** pytest.
- **Métricas de Cobertura:** Librería coverage.
- **Pruebas de Mutación:** Librería cosmic Ray.
- **Agente LLM:** Librería google-generativeai con model: "gemini-3.1-flash-lite"

Cada grupo podrá implementar internamente en su agente la lógica necesaria para ejecutar estas herramientas subprocesos, obtener los errores de importación/compilación o leer los puntajes de mutación para construir y refinar iterativamente los prompts enviados al modelo.

3. Parte II - Análisis y Presentación

La segunda etapa consiste en grabar un video explicativo de aproximadamente 5 minutos. Ambos integrantes deben demostrar brevemente el uso de su herramienta y enfocar la mayor parte del tiempo en responder de forma crítica y técnica a cuatro áreas clave:

1. **Desafíos Técnicos:** ¿Cuáles son las principales complicaciones y limitaciones prácticas que existen al momento de crear tests automatizados utilizando un LLM?
2. **Calidad del Código Generado:** ¿Qué tan buenos son los tests creados por su herramienta? Se debe analizar la cantidad y pertinencia de las aserciones, identificar si existen pruebas repetidas o inútiles, y explicar cómo los resultados de la herramienta de mutación ayudan a evaluar esto.
3. **Comportamiento Iterativo:** En la práctica, ¿cuántas iteraciones fueron necesarias en promedio para que el LLM generara una suite de pruebas válida? Explique por qué ocurren estos fallos en los primeros intentos.
4. **Algoritmo e Ingeniería de Prompts:** Explique la arquitectura y flujo de su agente junto con el diseño de los prompts. Discuta cómo interactúan y si existen mejores enfoques para estructurar esta solución.

4. Código Base Proporcionado

Para facilitar la configuración, se entrega Tarea1.zip con la siguiente estructura base:

```
Tarea1/  
|-- agent.py  
|-- requirements.txt  
|-- README.md  
|-- .env.example  
|-- cosmic-ray.toml  
|-- run_all.sh  
|-- Results/  
|   |-- .gitkeep  
|-- Public_Projects/  
|   |-- bridge/  
|       |-- __init__.py  
|       |-- action_event.py  
|       |-- dealer.py  
|-- gin_rummy/  
|   |-- __init__.py  
|   |-- base.py
```

Además, se incluye el script `run_all.sh`, el cual iterará sobre todos los proyectos públicos y sus respectivos archivos, ejecutando su agente de forma automatizada y guardando los resultados en la carpeta **Results**.

Para utilizarlo en terminales basadas en Unix (Linux, macOS o WSL), deben otorgarle permisos de ejecución y correrlo con los siguientes comandos:

```
chmod +x run_all.sh  
./run_all.sh
```

5. Restricciones

- No se debe modificar los proyectos publicos entregados.
- Sólo se permite utilizar la API de Gemini Flash Lite 3.1.
- No se permite utilizar otros modelos comerciales o cuentas pagadas.
- El agente debe ejecutarse de forma completamente automática iterando sobre sus propios hallazgos.
- No se permite modificar manualmente los tests generados antes de la entrega.
- No esta permitido colocar la grabación con velocidad aumentada (ejemplo : x2) para cumplir con el tiempo solicitado.

6. Plazos y Política de Atrasos

- **Fecha de inicio:** Martes 1 de septiembre de 2026.
- **Plazo de entrega (sin atraso):** Jueves 24 de septiembre de 2026 a las 23:59 hrs.
- **Uso de Cupones:** Cada estudiante cuenta con un número limitado de "cupones de atraso" para el semestre. El uso de **1 cupón** permite extender el plazo original en **24 horas** exactas sin penalización alguna en la nota. Los cupones se descuentan automáticamente del sistema de entregas.
- **Descuento por Atrasos:** Si se entrega la tarea tarde y ya no se dispone de cupones (o se agotan en la entrega), se aplicará un descuento estricto de **1.0 punto** (en escala 1.0 a 7.0) por cada 24 horas o fracción de atraso.
- **Límite Máximo:** No se recibirán tareas (y serán calificadas con nota 1.0) después de **2 días de atraso** respecto a la fecha de entrega original, sin importar si el alumno cuenta con cupones restantes o no.

7. Modalidad de Trabajo

Tal como fue informado previamente a través de los canales de anuncio del curso, el desarrollo de esta tarea debe realizarse en **parejas**.

8. Entregables

Si bien el trabajo es en parejas, basta con que **un solo integrante** del grupo suba los archivos a la plataforma. En caso de existir múltiples entregas por parte de ambos integrantes, **se revisará y calificará estrictamente el último envío realizado**.

- El código fuente completo del agente (.zip de la carpeta del proyecto).
- La suite de pruebas generada por el agente para los proyectos públicos entregados junto con la tarea.
- Un archivo README.md (máximo una página) describiendo brevemente la estrategia implementada, las principales decisiones de diseño y las limitaciones encontradas.
- Un enlace funcional y con los permisos adecuados para visualizar el video explicativo de 5 minutos.

9. Evaluación

La evaluación de la tarea se divide en un 60% para la codificación del agente y un 40% para el análisis en video.

9.1. Parte 1: Implementación del Agente (60%)

- **Funcionamiento de la Interfaz y Estructura (10%):** El agente se ejecuta correctamente respetando los parámetros de entrada. Utiliza Python 3.14 y la API de Gemini Flash Lite 3.1.
- **Cumplimiento de Ejecución Limpia (20%):** Los archivos generados compilan y pasan exitosamente las pruebas al correr pytest. *Nota:* Si un archivo generado no compila, se pierde este puntaje proporcionalmente y no se evaluará el siguiente ítem para ese archivo.
- **Logro de Métricas Automatizadas (30%):**
 - *Line Coverage (10%):* Promedio $\geq 80\%$ evaluado mediante coverage.
 - *Branch Coverage (10%):* Promedio $\geq 50\%$ evaluado mediante coverage.
 - *Mutation Score (10%):* Promedio $\geq 50\%$ evaluado mediante cosmic Ray.

9.2. Parte 2: Video y Análisis (40%)

- **Explicación del Algoritmo y Prompts (15%):** Explica claramente el flujo del agente, justifica el diseño de los prompts utilizados y la coordinación entre ambos para resolver el problema.
- **Análisis de Calidad y Complicaciones (10%):** Evalúa críticamente los límites del LLM y demuestra con ejemplos concretos de su código si las aserciones generadas son robustas o triviales.
- **Evaluación del Ciclo Iterativo (10%):** Presenta de forma clara los datos sobre los intentos promedio del LLM y justifica con argumentos técnicos la necesidad de iterar.
- **Presentación y Formato (5%):** El video respeta el límite de tiempo, posee buena calidad audiovisual y demuestra un dominio técnico acorde.